

UNIT 1 – BASH SHELL PRACTICAL

Study Answer Key • Questions 1–40 • What to memorize + command meaning + expected output

Quick study rule: Focus on the command pattern first. Important symbols/operators: **-d, -f, -e, -ge, -gt, -lt, \$!, \$\$, wait, |, &, mkfifo.**

Q1. Display current username, date, time and hostname

```
#!/bin/bash
echo "Username: $(whoami)"
echo "Date: $(date +%F)"
echo "Time: $(date +%T)"
echo "Hostname: $(hostname)"

# Remember:
# whoami = current user
# date = date/time
# hostname = computer name
```

Expected output / idea: Example output:

```
Username: khushi
Date: 2026-09-12
Time: 10:30:15
Hostname: ubuntu
```

Q2. Display present working directory and list all files/directories

```
#!/bin/bash
pwd
ls -la

# pwd = present working directory
# ls -la = list all files, including hidden files
```

Expected output / idea: Example output:

```
/home/khushi
... .bashrc file1.txt Documents ...
```

Q3. Accept directory name and change to it if it exists

```
#!/bin/bash
read -p "Enter directory name: " dir

if [ -d "$dir" ]; then
    cd "$dir"
    echo "Directory changed."
    pwd
else
    echo "Directory does not exist."
fi

# -d = checks whether path is a directory
```

Expected output / idea: Example:

```
Enter directory name: Documents
Directory changed.
/home/khushi/Documents
```

Q4. Create directory, enter it, create two empty files, display contents

```
#!/bin/bash
read -p "Enter directory name: " dir
mkdir -p "$dir"
cd "$dir"
touch file1.txt file2.txt
ls -la

# mkdir = create directory
# touch = create empty file
```

Expected output / idea: Example output:

```
file1.txt
file2.txt
```

Q5. Accept file name and check if it exists

```
#!/bin/bash
read -p "Enter file name: " file

if [ -e "$file" ]; then
    echo "File/path exists."
else
    echo "File/path does not exist."
fi

# -e = path exists
```

Expected output / idea: Example:

Enter file name: notes.txt

File/path exists.

Q6. Determine whether path is regular file, directory or other

```
#!/bin/bash
read -p "Enter path: " path

if [ -f "$path" ]; then
    echo "It is a regular file."
elif [ -d "$path" ]; then
    echo "It is a directory."
else
    echo "It is another type or does not exist."
fi

# -f = regular file
# -d = directory
```

Expected output / idea: Example:

Enter path: test.txt

It is a regular file.

Q7. Display all hidden files in current directory

```
#!/bin/bash
ls -d .[^.]*

# Hidden Linux files start with .
# [^.]* avoids showing . and ..
```

Expected output / idea: Example output:

.bashrc

.profile

.config

Q8. Count files and directories in current directory

```
#!/bin/bash
files=$(find . -maxdepth 1 -type f | wc -l)
dirs=$(find . -maxdepth 1 -type d | wc -l)

echo "Files: $files"
echo "Directories: $dirs"

# find = search
# wc -l = count lines
```

Expected output / idea: Example output:

Files: 5

Directories: 3

Q9. Display absolute home directory path and current working directory

```
#!/bin/bash
echo "Home directory: $HOME"
echo "Current directory: $(pwd)"

# $HOME = user's home directory
# pwd = current directory
```

Expected output / idea: The command/script performs the requested task.

Q10. Create Linux project directory structure with three subdirectories

```
#!/bin/bash
mkdir -p LinuxProject/Documents LinuxProject/Programs LinuxProject/Backup
ls -R LinuxProject
# mkdir -p = create parent + subdirectories if needed
```

```
# ls -R = recursive listing
```

Expected output / idea: Example:

LinuxProject/

■■■ Backup

■■■ Documents

■■■ Programs

Q11. Copy user-specified file to backup directory

```
#!/bin/bash
read -p "Enter file name: " file

if [ -e "$file" ]; then
    mkdir -p backup
    cp "$file" backup/
    echo "File copied to backup."
else
    echo "File does not exist."
fi

# cp = copy
```

Expected output / idea: Example:

Enter file name: notes.txt

File copied to backup.

Q12. Rename user-entered file

```
#!/bin/bash
read -p "Enter old file name: " old
read -p "Enter new file name: " new

if [ -e "$old" ]; then
    mv "$old" "$new"
    echo "File renamed."
else
    echo "File does not exist."
fi

# mv = move or rename
```

Expected output / idea: Example:

old: a.txt

new: b.txt

File renamed.

Q13. Move file to another directory and verify

```
#!/bin/bash
read -p "Enter file name: " file
read -p "Enter destination directory: " dir

if [ -e "$file" ] && [ -d "$dir" ]; then
    mv "$file" "$dir/"
    if [ -e "$dir/$file" ]; then
        echo "File moved successfully."
    fi
else
    echo "File or directory does not exist."
fi
```

Expected output / idea: Example:

File moved successfully.

Q14. Delete file only after checking it exists

```
#!/bin/bash
read -p "Enter file name: " file

if [ -e "$file" ]; then
    rm "$file"
    echo "File deleted."
else
    echo "File does not exist."
fi

# rm = remove/delete
```

Expected output / idea: Example:

Enter file name: old.txt

File deleted.

Q15. Accept two file names and check whether both exist

```
#!/bin/bash
read -p "Enter first file: " f1
read -p "Enter second file: " f2

if [ -e "$f1" ] && [ -e "$f2" ]; then
    echo "Both files exist."
else
    echo "Both files do not exist."
fi

# && = AND
```

Expected output / idea: Example:

Both files exist.

Q16. Display size, permissions, owner and modification details

```
#!/bin/bash
read -p "Enter file name: " file

if [ -e "$file" ]; then
    ls -lh "$file"
else
    echo "File does not exist."
fi

# ls -l = long details
# -h = human-readable size
```

Expected output / idea: Example:

-rw-r--r-- 1 khushi users 2.0K Sep 12 10:20 notes.txt

Q17. Display disk space usage and available space

```
#!/bin/bash
df -h

# df = disk filesystem information
# -h = human-readable
```

Expected output / idea: Example columns:

Filesystem Size Used Avail Use% Mounted on

Q18. Display memory usage and basic system information

```
#!/bin/bash
free -h
uname -a

# free -h = memory/RAM usage
# uname -a = system/kernel information
```

Expected output / idea: Example:

total used free

Mem: 7.7G 3.2G 1.1G

Linux ... x86_64 ...

Q19. Display logged-in users and current login user

```
#!/bin/bash
echo "Logged-in users:"
who

echo "Current user:"
whoami

# who = logged-in users
# whoami = current user
```

Expected output / idea: Example:

Logged-in users:

khushi pts/0 ...

Current user:

Q20. Display running processes and process ID of shell script

```
#!/bin/bash
echo "Shell script PID: $$"
echo "Running processes:"
ps

# $$ = PID of current shell/script
# ps = process status
```

Expected output / idea: Example:

Shell script PID: 2451

Running processes:

PID TTY TIME CMD

...

Q21. Accept PID and check whether process is running

```
#!/bin/bash
read -p "Enter PID: " pid

if ps -p "$pid" > /dev/null; then
    echo "Process is running."
else
    echo "Process is not running."
fi

# ps -p PID = check a particular process
```

Expected output / idea: Example:

Enter PID: 2451

Process is running.

Q22. Start a background process, display PID and wait

```
#!/bin/bash
sleep 5 &
pid=$!

echo "Background process PID: $pid"
wait "$pid"
echo "Process completed."

# & = run in background
# $! = PID of last background process
# wait = wait for process
```

Expected output / idea: Example:

Background process PID: 3012

Process completed.

Q23. Demonstrate positional parameters with three command-line arguments

```
#!/bin/bash
echo "First argument: $1"
echo "Second argument: $2"
echo "Third argument: $3"

# Run:
# ./script.sh Apple Mango Banana
# $1, $2, $3 = first, second, third arguments
```

Expected output / idea: Example:

First argument: Apple

Second argument: Mango

Third argument: Banana

Q24. Accept two integers and perform +, -, *, /

```
#!/bin/bash
read -p "Enter first integer: " a
read -p "Enter second integer: " b

echo "Addition: $((a+b))"
echo "Subtraction: $((a-b))"
echo "Multiplication: $((a*b))"
```

```
echo "Division: $((a/b))"

# $(( )) = arithmetic calculation
```

Expected output / idea: Example for 10 and 2:

Addition: 12
Subtraction: 8
Multiplication: 20
Division: 5

Q25. Check even or odd

```
#!/bin/bash
read -p "Enter number: " n

if [ $((n % 2)) -eq 0 ]; then
    echo "Even"
else
    echo "Odd"
fi

# % = remainder
# -eq = equal
```

Expected output / idea: Example:

Enter number: 8
Even

Q26. Find largest of three numbers

```
#!/bin/bash
read -p "Enter three numbers: " a b c

if [ "$a" -ge "$b" ] && [ "$a" -ge "$c" ]; then
    echo "$a is largest"
elif [ "$b" -ge "$a" ] && [ "$b" -ge "$c" ]; then
    echo "$b is largest"
else
    echo "$c is largest"
fi
```

Expected output / idea: Example:

10 25 15
25 is largest

Q27. Print numbers 1 to 10 using for loop

```
#!/bin/bash
for i in {1..10}
do
    echo "$i"
done

# for = repeat for each value
# {1..10} = 1 through 10
```

Expected output / idea: Example output:

1
2
3
...
10

Q28. Find sum of 1 to N using loop

```
#!/bin/bash
read -p "Enter N: " n
sum=0

for ((i=1; i<=n; i++))
do
    sum=$((sum+i))
done

echo "Sum = $sum"

# sum stores the running total
```

Expected output / idea: Example:

N = 5

Sum = 15

Q29. Display multiplication table

```
#!/bin/bash
read -p "Enter number: " n

for ((i=1; i<=10; i++))
do
    echo "$n x $i = $((n*i))"
done
```

Expected output / idea: Example for 5:

5 x 1 = 5

5 x 2 = 10

...

5 x 10 = 50

Q30. Find factorial of a number

```
#!/bin/bash
read -p "Enter number: " n
fact=1

for ((i=1; i<=n; i++))
do
    fact=$((fact*i))
done

echo "Factorial = $fact"
```

Expected output / idea: Example:

5! = 120

Q31. Check positive, negative or zero

```
#!/bin/bash
read -p "Enter number: " n

if [ "$n" -gt 0 ]; then
    echo "Positive"
elif [ "$n" -lt 0 ]; then
    echo "Negative"
else
    echo "Zero"
fi

# -gt = greater than
# -lt = less than
```

Expected output / idea: Example:

Enter number: -4

Negative

Q32. Accept student marks and display grade

```
#!/bin/bash
read -p "Enter marks: " marks

if [ "$marks" -ge 90 ]; then
    echo "Grade A"
elif [ "$marks" -ge 80 ]; then
    echo "Grade B"
elif [ "$marks" -ge 70 ]; then
    echo "Grade C"
elif [ "$marks" -ge 60 ]; then
    echo "Grade D"
else
    echo "Grade F"
fi

# if / elif / else = conditional decision
```

Expected output / idea: Example:

Enter marks: 85

Grade B

Q33. Use case menu for date/calendar/current directory/user

```
#!/bin/bash
echo "1. Date"
echo "2. Calendar"
echo "3. Current directory"
echo "4. Current user"
read -p "Enter choice: " ch

case $ch in
    1) date ;;
    2) cal ;;
    3) pwd ;;
    4) whoami ;;
    *) echo "Invalid choice" ;;
esac

# case = menu/multiple-choice decision
```

Expected output / idea: Example:

Choice 3
/home/khushi/Documents

Q34. Store 5 values in array and display elements and indexes

```
#!/bin/bash
arr=(10 20 30 40 50)

for i in "${!arr[@]}"
do
    echo "Index $i = ${arr[$i]}"
done

# array = stores multiple values
# ${!arr[@]} = array indexes
```

Expected output / idea: Example:

Index 0 = 10
Index 1 = 20
Index 2 = 30
Index 3 = 40
Index 4 = 50

Q35. Find word length

```
#!/bin/bash
read -p "Enter a word: " word
echo "Length = ${#word}"

# ${#word} = length of string
```

Expected output / idea: Example:

Enter a word: Linux
Length = 5

Q36. Demonstrate parent-like and child-like background tasks using subshells

```
#!/bin/bash
echo "Parent: starting child task..."

(
    echo "Child: task started"
    sleep 2
    echo "Child: task completed"
) &

child_pid=$!
wait "$child_pid"

echo "Parent: child finished."

# ( ... ) = subshell
# & = background
# wait = synchronize
```

Expected output / idea: Example order:

Parent: starting child task...
Child: task started
Child: task completed
Parent: child finished.

Q37. Run two commands in separate subshells and wait for both

```
#!/bin/bash
(
    echo "Task 1 started"
    sleep 2
    echo "Task 1 completed"
) &
pid1=$!

(
    echo "Task 2 started"
    sleep 3
    echo "Task 2 completed"
) &
pid2=$!

wait "$pid1"
wait "$pid2"

echo "Both tasks completed."

# Two subshells run in background
# wait synchronizes them
```

Expected output / idea: Example:

```
Task 1 started
Task 2 started
Task 1 completed
Task 2 completed
Both tasks completed.
```

Q38. Demonstrate IPC using an unnamed pipe

```
#!/bin/bash
echo "Hello from first command" | cat

# | = pipe
# Output of the command on the left becomes
# input to the command on the right.

# Simple exam demonstration:
# echo "message" | cat
```

Expected output / idea: Example output:

```
Hello from first command
```

Q39. Create FIFO, send message, read it and remove FIFO

```
#!/bin/bash
fifo="myfifo"

mkfifo "$fifo"

echo "Hello through FIFO" > "$fifo" &
cat "$fifo"

rm "$fifo"
echo "FIFO removed."

# mkfifo = create named pipe
# cat = read
# rm = remove
```

Expected output / idea: Example:

```
Hello through FIFO
FIFO removed.
```

Q40. Run child task in background, capture exit status with wait

```
#!/bin/bash
(
    echo "Child task running..."
    sleep 2
    exit 0
) &

pid=$!
wait "$pid"
status=$?
if [ "$status" -eq 0 ]; then
```

```
        echo "Child task completed successfully."
    else
        echo "Child task failed."
    fi

    # wait returns the child's exit status
    # 0 = success
```

Expected output / idea: Example:

Child task running...

Child task completed successfully.